

PROGRAMACIÓN CONCURRENTE Y PARALELA

Práctica 2: Semáforos

1er Cuatrimestre 2026

Para esta práctica, asumimos que la operación `print` es atómica.

Ejercicio 1. Dados los siguientes threads

```
thread T1 {                thread T2 {
    print(A)                print(E)
    print(B)                print(F)
    print(C)                print(G)
}
```

Agregar semáforos en el programa para garantizar que en toda ejecución se cumpla que A se muestra antes que F y que F se muestra antes que C

Ejercicio 2. Dados los siguientes threads

```
thread T1 {                thread T2 {
    print(C)                print(A)
    print(E)                print(R)
                                print(O)
}
```

Utilizar semáforos para garantizar que las únicas salidas posibles sean ACERO y ACREO.

Ejercicio 3. Dados los siguientes threads

```
thread T1 {                thread T2 {                thread T3 {
    print(R)                print(I)                print(O)
    print(OK)                print(OK)                print(OK)
}
```

Utilizar semáforos para garantizar que el único resultado impreso será R I O OK OK OK.

Ejercicio 4. Dados los siguientes threads

```
thread T1 {                thread T2 {                thread T3 {
    while (true) {          while(true) {          while (true) {
        print(A)              print(E)              print(H)
        print(B)              print(F)              print(I)
        print(C)              print(G)
        print(D)
    }
}
```

Agregar semáforos para garantizar, simultáneamente, que

- la cantidad de Fs sea menor o igual a la cantidad de As,
- la cantidad de Hs sea menor o igual a la cantidad de E, y
- la cantidad de Cs sea menor o igual a la cantidad de G.

Ejercicio 5. Considere los siguientes dos procesos

```

thread T1          thread T2
while (true)      while (true)
    print(A)        print(B)
    
```

- a) Utilizar semáforos para garantizar que en todo momento la cantidad de As y Bs difiera al máximo en 1.
- b) Modifique la solución para que la única salida posible sea A B A B A B A B . . .
- c) Modifique la solución para que la única salida posible sea A B B A B B A B B . . .

Ejercicio 6. Se busca computar la suma de los primeros N números impares utilizando dos threads llamados **generador** y **acumulador**, que deben cooperar entre sí para resolver esta tarea. Contamos con dos variables globales **impar**, que comienza inicializada en N , y **suma**, que comienza inicializada en 0. Buscamos que **generador** se encargue de que **impar** contenga el valor i cuando se deba sumar el i -ésimo número impar, y que **acumulador** compute cuál es el i -ésimo número impar lo acumule en **suma**. Al finalizar el cómputo, el thread **generador** debe imprimir el valor correcto de la sumatoria. Implemente este programa en Java utilizando semáforos como mecanismo para sincronizar a **generador** y **acumulador**.

Ejercicio 7. En un gimnasio hay cuatro aparatos, cada uno permite trabajar un grupo muscular distinto. Los aparatos se cargan con discos (todos del mismo peso). Cada cliente del gimnasio posee una rutina que le indica qué aparatos usar, en qué orden y cuántos discos de peso utilizar en cada caso. La rutina no tiene una longitud fija y puede implicar repetir o no usar algún aparato. Por otra parte, como norma, el gimnasio exige que cada vez que un cliente termina de utilizar un aparato descargue todos los discos que usó y los coloque en el lugar destinado a su almacenamiento (aún si piensa volver a usar el mismo aparato).

- a) Piense un modelado para este problema e indique los recursos compartidos y agentes activos en el mismo.
- b) Provea un código que simule el funcionamiento del gimnasio, garantizando exclusión mutua en el acceso a los recursos compartidos y que esté libre de deadlocks y livelocks. Implemente su solución en Java.
- c) Indique si su solución está libre de inanición. En caso contrario, explique cómo podría resolverlo.

Ejercicio 8. Considere el siguiente juego: existen dos tipos de participantes, los *generadores* y los *consumidores* de bolitas. Los generadores crean bolitas de a una a la vez y las colocan en una bolsa (suman un punto por cada bolita que logran colocar en la bolsa). Los consumidores toman

pares de bolitas y suman un punto por cada par (pero no pueden tomar mas de un par por vez). No se conoce a priori la cantidad de participantes que están jugando, pero se debe garantizar en todo momento que la bolsa no contenga mas bolitas que la cantidad de generadores que se encuentran jugando.

Proponga una solución basada en semáforos que sea libre de deadlocks para todo escenario en donde participen 2 o mas generadores.

Ejercicio 9. Para cruzar un determinado río, se tiene un bote transbordador con capacidad para N pasajeros. Las personas van llegando a alguna de las dos costas del río y deben esperar a que el bote esté disponible para que puedan subir. Cuando le es posible, cada persona debe acomodarse en un asiento y luego esperar a que el bote haya llegado a la costa opuesta. La dinámica en la que funciona el bote es la siguiente:

- Empieza en la costa *oeste* (podemos pensar que es la costa “0”).
- Espera en la costa hasta que haya gente acomodada en los N asientos.
- Viaja hasta la costa *este* (“1”).
- Amarra, permitiendo que los pasajeros descendan.
- Repite el procedimiento desde el principio, en la costa actual.

Modele el escenario descrito utilizando un thread **transbordador** para el bote y uno **persona** para cada persona que llegue a una costa. Modele las esperas a realizar mediante el uso de semáforos. Tenga en cuenta que el thread **persona** puede llevar un parámetro que indica en qué costa comienza su viaje. Se proponen dos variantes:

- a) Cuando el bote llega a una costa, espera que *todos* sus ocupantes terminen de bajar antes de permitir subir a la gente que está esperando allí.
- b) Cuando el bote llega a una cosa, la gente empieza a bajar y subir en forma concurrente, sólo se busca que no hayan más de N personas a bordo en un momento dado.

Ejercicio 10. Se desea modelar una planta de refinamiento donde vehículos autónomos trasladan productos en distintos estados de procesamiento. La planta consta de una plataforma donde se recibe la materia prima, otra donde se depositan los productos terminados, 8 máquinas procesadoras y 4 vehículos.

Cada máquina procesadora es capaz de descargar el contenido de un vehículo, realizarle algún procesamiento y cargar el producto refinado en un vehículo. No necesariamente se carga el producto procesado en el mismo vehículo del cual se descargó la materia prima.

Cada vehículo posee una lista ordenada de cargas y descargas que debe efectuar en distintas máquinas o plataformas. Las acciones de carga, descarga, procesamiento y desplazamiento de los vehículos consumen un tiempo no despreciable. Por esto, es importante garantizar que los vehículos permanecen junto a las máquinas mientras se realizan las acciones de carga y descarga. Además, las acciones de desplazamiento y procesamiento deben poder realizarse de manera concurrente.

Modele este escenario utilizando semáforos, asumiendo que:

- siempre es posible realizar una acción de carga en la plataforma de recepción y siempre es posible descargar en la plataforma de entrega,

- ninguna ruta requiere descargar en la plataforma de recepción ni cargar en la de entrega, y
- un vehículo esperando a ser cargado no obstruye a un vehículo siendo descargado por la misma máquina.

Ejercicio 11. En una oficina hay un baño sin distinción de género con 8 toiles. A lo largo del día, distintas personas entran a utilizarlo. Si sucede que en ese momento todos los toiles están ocupados, las personas esperan hasta que alguno se libere. Por otra parte, periódicamente el personal de limpieza debe pasar a mantener las instalaciones en condiciones. La limpieza del baño no se puede hacer mientras haya gente dentro del mismo, por lo que si en ese momento hay personas utilizando algún toilete o esperando que se libere alguno, el personal de limpieza debe esperar a que el baño se vacíe completamente. En contraparte, si hay un empleado de limpieza trabajando en el baño, las personas que quieran utilizarlo deberán esperar a que termine.

- a) Modele esta situación utilizando semáforos como mecanismo de sincronización (puede modelar al personal de limpieza como un thread único).
- b) Modifique la solución anterior para contemplar el caso donde el personal de limpieza tiene prioridad. Es decir, si hay un empleado de limpieza esperando para hacer el mantenimiento, las siguientes personas que lleguen deben esperar a que logre terminar la limpieza.
- c) Modifique la solución para que la gente que está esperando que se libere un toilete *no* sea considerada para la espera del personal de limpieza, es decir, al llegar esperan que la gente que ya está usando un toilete termine pero luego comienzan la limpieza aunque hubiera gente de antes esperando que se desocupara alguno.

Ejercicio 12. Se desea modelar el control de tránsito de un puente que conecta dos ciudades. Dado que el puente es muy estrecho se debe evitar que dos autos circulen al mismo tiempo en dirección opuesta, dado que quedarían atascados. Resuelva los siguientes problemas usando semáforos, modelando cada coche como un thread independiente que desea atravesar el puente en alguna de las dos direcciones posibles (tenga en cuenta que atravesar el puente no es una acción atómica, y por lo tanto, requiere de cierto tiempo),

- a) De una solución que permita que varios coches que se desplazan en la misma dirección puedan circular simultáneamente.
- b) Modifique la solución anterior para que como máximo 3 coches puedan circular por el puente al mismo tiempo.
- c) Indique si la solución propuesta en el punto b es libre de inanición. Justifique su respuesta.